Build an interactive app that does the following with the title and abstract of the grants:

1. Clean up the text from boilerplate terms (e.g., CAREER, intellectual merit)
2. Transform them into embedding vectors using the SPECTER2 model (with the proximity option)
3. Uses UMAP to create a 2D projection
4. Create an interactive visualization tool that visualizes the UMAP output

The visualization should be designed as follows:

1. Use a white background UI
2. Color the grants according to their funding source (this means you have to keep this metadata when extracting information from the grants table)
3. Include an interactive option that can be toggled to scale the grants by their amount

Show me a plan before executing it.

Show less

I'll take a look at the actual data first so the plan is grounded in what's really in these files.

Ran 5 commands
Ran 5 commands

Here's what's in the data and the plan that follows from it. Two things surfaced that shape the whole approach, so I want to flag them before building anything.

What the data actually contains

The `grants.csv` has 2,676 grants. All have a title (`grantname`); 1,928 have an abstract. Amount is in `totaldollars` (ranges from ~$500 to ~$38.6M — very skewed, so scaling will need a sqrt or log transform, consistent with your usual perceptual-scaling approach).

Two findings that change the plan:

The `funding_source` column is empty for all 2,676 rows. The real funding-source signal lives in `agencycode`: NSF (1686), NIH (546), NIH-SUB (105), Navy (87), NASA (49), Army (48), DOE (37), AFRO (31), then a long tail. So "color by funding source" will use `agencycode` (with `agencyname` kept as the readable label). I'd propose folding NIH-SUB into NIH and bucketing the rare agencies (<~15 grants) into "Other" so the palette stays legible — but that's a decision point for you.

SPECTER2 and UMAP cannot run in this container. Network egress is disabled (HuggingFace returns 403), and `torch`, `transformers`/`adapters`, and `umap-learn` are all

absent with no way to install them here. This isn't a soft limitation — the model weights and the proximity adapter can't be fetched. That's actually fine architecturally, because you never run SPECTER2/UMAP live in a browser anyway; embeddings and projection are always a precompute step. So the real design splits cleanly:

Proposed architecture

Part A — `pipeline.py` (runs on your machine, where you have network + can pip install):

1. Text cleaning: strip mechanism/boilerplate tokens. From the scan, the worth-removing terms are program-name prefixes (CAREER: 158, Collaborative Research: 386, RAPID/EAGER/CRII/REU/RUI/SBIR/STTR, "Conference:/Workshop:/Travel:/Symposium:") and NSF abstract boilerplate ("Intellectual Merit" / "Broader Impacts" appear in 808 abstracts), plus the leading grant-ID/PI-name stub some NSF abstracts start with. Cleaned text = cleaned title + `[SEP]` + cleaned abstract (title-only for the 748 without abstracts).
2. SPECTER2 via `allenai/specter2_base` + the **proximity** adapter loaded through the `adapters` library (this is the officially recommended proximity setup).
3. UMAP → 2D.
4. Writes `grants_umap.json`: `{id, x, y, title, agency, agencyLabel, amount, year, hasAbstract}`.

I'll write this as a complete, ready-to-run script and validate the text-cleaning portion here (it needs no special libraries), so the only thing you supply is compute.

Part B — the interactive app (`index.html`, built and working today):
Vanilla HTML + D3 + canvas (for 2,676 points), white UI, loading `grants_umap.json`. Features: color by funding source with a legend that doubles as a filter, a **toggle to size points by `totaldollars`** (sqrt-scaled; uniform-size when off), hover tooltips (title, agency, amount, year), zoom/pan, and a search box. Same self-contained pattern you've been using.

Bridging the container limitation: so the app is genuinely functional *now* rather than an empty shell, I'll generate a real 2D projection offline using only sklearn (TF-IDF → TruncatedSVD → t-SNE — all installed and verified working). These are honest, meaningful coordinates from the actual cleaned text; they're just not SPECTER2/UMAP. The JSON schema is identical, so when you run `pipeline.py` you overwrite one file and the app shows the real embedding with zero code changes. I'll label the preview coordinates clearly so there's no confusion about which projection is loaded.

The one fork I'd like your call on

Do you want me to build with the sklearn preview coordinates now (app works end-to-end today, you swap in real SPECTER2+UMAP output later), or hold the visualization until you've run the real pipeline and just deliver `pipeline.py` first?

And on the two smaller decisions: fold NIH-SUB→NIH and bucket rare agencies into "Other"? Embed the 748 abstract-less grants as title-only, or drop them?

- Yes, build a sklearn preview
- Do not fold NIH sub-awards
- Group small funding sources into an "other" category
- Drop grants without absract

Ran a command
Ran a command

Good — that changes the picture usefully: on the abstract subset, NIH-SUB/Navy/Army/AFRO have zero abstracts and drop out entirely, leaving NSF (1524), NIH (354), then a small tail. I'll keep agencies with ≥10 grants named (NSF, NIH, NASA, DOE, NEH) and bucket USDA + NOAA into "Other" — a clean 6-color palette, with the threshold as an adjustable constant.

Let me check the design skill, then build.

Viewed a file
Viewed a file

Design is clear: this is an exploratory scientific instrument for a viz researcher, so the scatter is the hero and the chrome stays quiet and precise — white canvas, a colorblind-safe qualitative palette (no red, per your usual discipline), legend-as-filter, and a clean amount-scaling toggle. Let me build the pieces.

Created 2 files, ran 2 commands
Created 2 files, ran 2 commands

Preview coordinates look good (1928 points, clean spread, boilerplate stripped, 464 KB — fine to embed inline). Now the app itself.

Created a file, ran a command
Created a file, ran a command

Now the real pipeline script for your machine (SPECTER2 proximity + UMAP), producing the identical JSON schema so the app swaps in with no code change.

Created a file, ran a command, read a file
Created a file, ran a command, read a file

Built to your four decisions. Here's what each piece does and the two things worth knowing.

`grant_atlas.html` — the app, self-contained (D3 from CDN, 1,928 grants embedded inline). White UI, points colored by funding source, legend rows double as show/hide filters, hover tooltips (title, agency, year, amount), scroll-zoom / drag-pan with reset, and a title search that highlights matches and dims the rest. The **"Size by amount" toggle** sqrt-scales radius by `totaldollars` (your usual perceptual scaling) and reveals a size key; off = uniform dots. The header and footer both flag that these are t-SNE preview coordinates.

`pipeline.py` — the real SPECTER2 (proximity adapter) + UMAP run for your machine. It writes a `grants_umap.json` with the identical schema, so you drop it next to the HTML, re-embed, and the header flips to "SPECTER2 + UMAP" on its own — no viz changes. It imports **`clean_text.py`**, the shared cleaner, so the text fed to the real embedding is byte-identical to the preview. **`build_preview.py`** is the sklearn preview generator I ran here, kept so the two paths stay in sync.

Two notes on your decisions:

The named/Other split fell out cleaner than expected once no-abstract grants were dropped: **NIH sub-awards, Navy, Army, and AFRO have zero abstracts**, so they disappear entirely — the NIH-vs-NIH-SUB question is moot on this subset. What remains is NSF (1524), NIH (354), NASA (16), DOE (14), NEH (13), and Other = USDA + NOAA (7). The threshold is a single constant (`NAMED_MIN = 10`) in both scripts if you want to move it.

On cleaning, I strip stacked program prefixes (CAREER:, Collaborative Research:, RAPID:, etc.), the leading NSF grant-id/PI stub, and the "Intellectual Merit / Broader Impacts / statutory mission" boilerplate — conservatively, leaving scientific content intact. The removal list lives at the top of `clean_text.py`; easy to extend if you spot mechanism terms I missed once you see the map.

One thing to flag honestly: the preview is TF-IDF → SVD → t-SNE, which keys on lexical overlap, so its clusters reflect shared vocabulary. SPECTER2 keys on citation-informed semantics, so expect the real map to regroup meaningfully — treat the current layout as a working harness for the interaction design, not as the final semantic structure.

Grant atlas
Code · HTML
Pipeline
PY
Clean text
PY
Build preview
PY

Update the grants atlas so that the aspect ratio of the plot stays the same even when I change the dimensions.

The distortion comes from the plot's x and y scales being fit independently to the canvas width and height, so the map stretches with the container. The fix is a single uniform pixels-per-data-unit factor for both axes, with the map centered in whatever space is available.

Ran 2 commands, edited a file
Ran 2 commands, edited a file

The earlier rebuild never ran (a shell-syntax error aborted the whole line before Python executed). Rebuilding now:

Ran a command, read a file
Ran a command, read a file

Done. The plot now uses a single pixels-per-data-unit factor for both axes (`s = min(availWidth/dataW, availHeight/dataH)`) and centers the map in whatever space is left over. Resizing the window rescales the whole embedding uniformly instead of stretching it, so distances stay faithful — which matters for a map where proximity is the whole point. Zoom already scaled both axes equally, so the aspect ratio holds through zoom and pan too. When you swap in the real SPECTER2 + UMAP coordinates, this applies automatically since it keys off the data extent.

Grant atlas
Code · HTML

Refine the UI so that the menus can be hidden/folded on the left side of the UI

I'll make the whole control dock fold away to the left edge, leaving a small opener button to bring it back — keeping the chrome quiet and the map unobstructed when collapsed. Let me view the exact markup first, then edit the template.

Viewed 3 files, edited a file
Viewed 3 files, edited a file

Now the markup — add the fold bar inside the dock and the opener button in the stage:

Edited 2 files
Edited 2 files

Now wire up the toggle logic in the boot section:

Edited a file, ran a command, read a file
Edited a file, ran a command, read a file

The control panel now folds. A "Controls" bar sits at the top of the dock with a fold button (‹) — click it and the whole panel slides off the left edge, uncovering the full map. A small sliders button appears top-left to bring it back. Focus moves to whichever control is relevant after each

toggle, and Escape folds the panel when you're not typing in the search box. The transition respects reduced-motion, and on mobile the full-width dock slides away cleanly too.

Grant atlas
Code · HTML

Add a slider that permits highlighting the grants by year. It should work as follows:

1. When filtering by time is on the grants with excluded dates, they are just grayed out, not removed.
2. The slider should allow me to choose the minimum and maximum years, as well as drag a year range (e.g., 5 years) across the full range of years.

This calls for a dual-thumb year slider where the band between the thumbs is itself draggable (so you can slide a fixed window across the timeline), plus a toggle that grays out-of-range grants rather than removing them. Let me look at the current draw loop and toggle CSS so the graying integrates cleanly.

Viewed a file, edited a file
Viewed a file, edited a file

Now the CSS for the slider, added after the size-key styles:

Edited a file
Edited a file

Now the Year card markup, inserted after the funding-source legend card:

Edited a file
Edited a file

Now the year state and slider logic. First the state variables (must exist before the first `draw()`):

Edited 2 files, ran a command, read a file
Edited 2 files, ran a command, read a file

Added a "Filter by year" card. Flip the toggle on and the dual-thumb slider activates (data spans 1995–2026). It works three ways: drag either handle to set the earliest or latest year, drag the blue bar between them to slide a fixed-width window across the timeline (so a 5-year band stays 5 years as you move it), or click anywhere on the track to jump the nearer handle. The readout shows the current window, and arrow keys nudge a focused handle by one year.

Grants outside the window are grayed — light gray at low opacity, drawn beneath the in-range points, never removed — so you keep the full-map density as context while the selected era stays in full color. It composes with the other controls: legend hiding still removes a source entirely, and search highlighting still works within whatever's grayed or not. When the toggle is

off, the slider is inert and no graying is applied. (There are no missing years in this dataset, but null-year grants would gray out when the filter is on.)

Grant atlas
Code · HTML

It looks like we stopped having NIH grants in 2021? This is weird. Can you check the data please?

Ran a command
Ran a command

That explains it — and it's a data artifact, not reality. Let me confirm whether it's NIH-specific.

Ran a command
Ran a command

Good catch, and it's a data artifact rather than a real trend. NIH didn't stop — the *abstracts* did.

The atlas only plots grants that have an abstract (that was your "drop grants without abstract" decision). NIH grant records continue right through 2025, but their abstracts vanish after 2019:

Year	NIH grants (total)	with abstract (plotted)
2019	28	18
2020	25	1
2021	29	0
2022	38	0
2023	27	0
2024	23	0
2025	16	0

So there are 133 NIH grants in 2021–2025 that simply have no abstract text in `grants.csv` (the `db_coverage` field is blank for all of them too), and every one gets filtered out before plotting. The cliff at ~2020 is when NIH abstract coverage in this dataset ends.

It's specific to NIH. NSF keeps essentially full abstract coverage the whole way through (85/85 in 2021, 86/86 in 2024), which is why NSF still populates recent years and NIH looks like it flatlined. That likely reflects how the data was sourced — NSF publishes award abstracts openly, whereas recent NIH abstracts weren't ingested here.

A couple of ways to handle it, depending on what you want the map to say:

The honest-scope option is to leave the embedding as-is (it's a semantic map of abstract text, so it can only include grants with text) but note the coverage gap in the UI so the missing recent NIH isn't misread as a funding decline. The fuller-coverage option is to embed those 133 NIH grants title-only — SPECTER2 handles title-only inputs fine, and it'd restore NIH presence in recent years, at the cost of mixing title-only and title+abstract embeddings (the title-only ones tend to cluster a bit differently since they carry less signal).

Want me to pull the recent NIH grants back in as title-only, or add a coverage note to the atlas instead?

Try to include the ones with title only even if it's not optimal without using a phrase embedder.

Right — "include title-only" without a phrase embedder just means stop dropping grants that lack an abstract and let the bare title be the document, running through the same pipeline (TF-IDF now, SPECTER2 title-only later) rather than routing short text to a separate model. Let me look at what including everything does to the agency mix, since that drives the palette.

Ran a command
Ran a command

Including everything adds back 748 title-only grants and restores NIH through 2025. It also brings in a middle tier of agencies that previously had no abstracts at all (Navy, Army, AFRO, NIH-SUB, HHS). To keep the palette legible I'll name the eight best-populated sources and sweep the long tail into "Other" (raising the `NAMED_MIN` threshold to 30), keeping NIH sub-awards as their own category per your earlier instruction. Let me update both scripts and the palette.

Edited 2 files
Edited 2 files

Now the same changes in the real pipeline:

Edited 2 files
Edited 2 files

Now expand the palette in the app template:

Edited 2 files, ran a command
Edited 2 files, ran a command

NIH now runs unbroken through 2025. Let me add a subtle "title only" marker to the tooltip so the lower-fidelity points are identifiable, then rebuild.

Edited a file, ran a command, read a file

Edited a file, ran a command, read a file

All 2,676 grants are now in the map (748 of them title-only), and NIH runs unbroken 1995–2025. The title-only grants go through the identical pipeline on their cleaned title alone — TF-IDF here, `title + [SEP]` (empty abstract) in `pipeline.py` for the real SPECTER2 run — so no separate phrase embedder is involved.

Because abstract-less agencies rejoined, the funding-source set grew. I named the eight best-populated sources and grouped the tail into "Other":

NSF (1686), NIH (546), NIH-SUB (105, kept separate per your earlier call), Navy (87), NASA (49), Army (48), DOE (37), AFRO (31), and Other (87 — HHS, NEH, NOAA, USDA, and singletons). The palette is a red-free, roughly colorblind-aware qualitative set; the threshold is a single constant (`NAMED_MIN = 30`) if you want to break out more or fewer.

Two things worth flagging. NEH dropped out of its own color into "Other" — it has 19 grants, below the threshold — so if you want humanities visible as a distinct hue again, lower `NAMED_MIN` to ~19 and I'll add a color. And since you flagged the fidelity tradeoff, hovering a title-only grant now shows a small "title only" tag; expect those 748 points to sit a bit differently once the real SPECTER2 embedding runs, since a title carries much less signal than a title-plus-abstract — the t-SNE preview leans on lexical overlap, so short titles cluster more loosely.

Grant atlas
Code · HTML
Pipeline
PY
Build preview
PY

Add a new element to the pipeline: run LDA on the titles+abstract (only title for the grants without an abstract) and add an option to the interface to color by topic rather than funding source. Before moving forward, let's brainstorm how to choose K for LDA.

Good instinct to settle K first — for this particular use it's a more loaded choice than usual, because K isn't only a modeling parameter here, it's also your color budget. Let me lay out the axes and then give you an opinionated path.

The reframe worth leading with: the standard advice ("scan K, maximize coherence") almost always pushes you toward a larger K than you can actually color. On a corpus like this, C_v coherence typically keeps climbing into the 20s–40s before plateauing, but a categorical color scale falls apart past ~10–12 hues, especially on a dense scatter where NSF already dominates the mass. So the coherence-optimal K and the legible-color K are usually different numbers, and I think the right move is to treat legibility as the binding constraint and use the quantitative work to validate quality within that ceiling rather than to find a global optimum.

On the quantitative side, the options in rough order of usefulness here: C_v coherence (Röder et al.) scanned over a grid is the workhorse — look for an elbow, not a razor peak. Perplexity / held-out likelihood I'd mostly ignore for K selection; it monotonically improves and Chang et al.'s "reading tea leaves" result is that it anti-correlates with human interpretability. Worth adding two cheap diagnostics: topic-size distribution (reject K values that produce one giant catch-all topic plus a litter of tiny ones) and topic diversity (fraction of unique words across the top-N term lists — low diversity means topics are bleeding into each other, a sign K is too high for the vocabulary). And because LDA is stochastic, run 3–5 seeds at each candidate K and check that the top-term lists are stable across seeds; an unstable K isn't a topic scheme you want to color by.

The qualitative check is non-negotiable given the whole point is an interpretable legend — you'll want to read the top ~10 terms per topic for the two or three finalist K's and confirm each topic is nameable. This is also where the two data caveats bite: the 748 title-only docs carry little signal and tend to smear toward generic topics, and short-doc LDA generally favors fewer, broader topics. Both argue for the lower end of the range.

Two ideas specific to your setup that I'd lean on. First, a domain prior: you know roughly how many broad research areas span these colleges (Khoury, COE, COS, CAMD, health, etc.) — that's a defensible anchor, probably ~8–12, and it makes the topics legible against your co-PI/college work. Second — and this is the nice one — you already have the UMAP layout, so the atlas is its own diagnostic: color the scatter by LDA dominant topic and see whether topics form spatially coherent regions on the map. If the right K is chosen, topic colors should pool into contiguous areas of the embedding; if they're sprayed randomly across it, K or the preprocessing is off. That's a viz-native validation you can't get from coherence numbers alone.

If you want to decouple the two constraints entirely, we can fit LDA at a coherence-good K (say 15–20) and then color by only the top ~8 topics with everything else folded into a gray "mixed/other," or group topics into macro-clusters for color while keeping fine topics in the tooltip. That gets modeling quality and a legible palette at once.

My recommended procedure: scan K ∈ {6, 8, 10, 12, 15, 20} on the cleaned title+abstract text, compute C_v + diversity + seed-stability, shortlist two or three, read their top terms, and confirm with the spatial-coherence check on the atlas — expecting to land around 8–12, with the color cap as the tiebreaker.

One question before I build: do you want K to *be* the number of colors (pick a legible K directly, simplest), or should I fit at a higher coherence-driven K and map down to a capped palette (better topics, slightly more machinery)? And should I anchor the grid around your sense of the college/field count, or scan blind and let coherence speak first?

After reading I changed my mind about coloring. Do not color by topic. The best is to create a list of topic and then in the UI organize it so that when I hover a topic in the list the dots in the atlast are highlighted by graying out the rest. What do you think?

I think it's the better design, and not just a lateral move. Coloring by topic forced two unrelated variables (funding source and topic) to compete for the one color channel, and it capped K at whatever the palette could hold. Your version cleanly separates them: color stays funding source, and topic becomes a *selection* layer expressed through the gray-out affordance you already have from the year filter. That consistency is worth a lot — the atlas gets a second lens without a second visual language, and K is now free to be whatever the topics want to be rather than whatever the palette tolerates. So the whole "K vs. color budget" tension from the last turn dissolves; we pick K purely on interpretability, and can comfortably go higher (12–15 is realistic now).

A few refinements I'd fold in:

Make membership graded, not binary. LDA already gives every grant a distribution over topics, so instead of "in topic / not in topic," set each dot's emphasis to its p(topic | doc) when a topic is active — strong members render solid, weak ones fade toward gray. It's a truer picture of a soft model and it reads beautifully on a scatter (you see a topic's core and its penumbra). Binary-by-dominant-topic is the fallback if you'd rather keep it crisp.

Pair hover with click-to-pin. Pure hover is great for fast scanning but you lose the highlight the moment you move toward the atlas to inspect points. Hover = transient preview, click = pin it so you can go read the dots; click again or click empty space to release.

Let the list carry its own labels. Each topic row shows its top ~4 terms as the name (auto-generated, you rename later) plus its size; sort by size descending. The list is scrollable inside the dock, and it composes with the existing filters — a dot stays lit only if it's in the pinned topic *and* in the year window *and* an active funding source. I'll also add the dominant topic to the hover tooltip so the relationship reads both directions.

On K itself: since I can't reach gensim offline, I'll run the scan with sklearn LDA and intrinsic (corpus-based UMass) coherence plus topic-diversity and size-distribution, across $K \in \{8, 10, 12, 15, 20\}$, and print the top terms per topic so you can actually read them. Then the atlas doubles as the final check — a good K should make topic highlights pool into contiguous regions of the UMAP rather than spray across it. The 748 title-only docs will still smear toward generic topics; that pushes slightly lower.

Two calls before I build: graded (weighted) highlighting or binary dominant-topic? And should I run the K scan now and bring you the candidate topic lists to pick from, or do you already have a K in mind?

- Weighted highlighting
- Run a scan to choose K
- In the UI give a name to the topics in the list and then add an option to see the keyword on demand

This design lets K float free, so I'll run the scan properly and pick on interpretability. Let me build the LDA module and the K-scan, run it, read the topics, then wire the interface.

Created 2 files, ran 2 commands
Created 2 files, ran 2 commands

The scan points to K=12 (best coherence −1.677, good diversity, best stability of the higher K's), but there's obvious noise: `andgt`/`andlt` are showing up as top terms — mangled HTML entities in the abstracts. Let me check and fix the cleaner before locking K.

Ran a command
Ran a command

The abstracts contain HTML markup where </>/& were mangled into `andlt`/`andgt`/`andamp` (≈2,100 occurrences each) — pure noise dominating the topics. Let me scrub markup in the cleaner.

Edited 3 files, ran a command
Edited 3 files, ran a command

Markup noise is gone and coherence improved. K=12 is the pick: best coherence (−1.612, tied with 10 but with better-separated topics — it splits ML from systems/security), good diversity (0.87), balanced sizes (no catch-all), and the topics are all cleanly nameable. K=15 starts fragmenting (two materials topics, split health) with lower coherence and stability. Now that color budget is off the table, 12 is comfortable for a list.

Let me add topic naming and metadata to the shared module, then wire it into both scripts.

Edited a file
Edited a file

Now add the LDA step to the preview build, attaching per-doc topic weights and writing topic metadata:

Edited 2 files, viewed 2 files, ran a command
Edited 2 files, viewed 2 files, ran a command

All 12 topics named correctly. Now add the same LDA step to the real pipeline:

Viewed 2 files, edited 2 files
Viewed 2 files, edited 2 files

Now the interface. First the topics-list styling:

Edited a file
Edited a file

Now the Topics card markup, after the Year card:

Edited a file

Edited a file

Now the data injection point for topics:

Edited a file
Edited a file

Now the core: rewrite `draw()` for weighted topic highlighting composed with the other filters. Let me view the current version.

Viewed 2 files, edited a file
Viewed 2 files, edited a file

Now add topic state, the list builder, and interactions. First the state:

Edited a file
Edited a file

Now the tooltip — add the dominant topic line:

Edited a file
Edited a file

Now update the zoom handlers to track drags (so a click on empty space can release a pin), and add the topic-list logic:

Edited 3 files, ran 2 commands, read a file
Edited 3 files, ran 2 commands, read a file

Done. Here's where it landed.

Choosing K. The scan (`scan_k.py`, over K ∈ {8,10,12,15,20} with UMass coherence, topic diversity, seed-stability, and size spread) surfaced a data problem first: the abstracts carried mangled HTML (`andlt`/`andgt`/`andamp`, ~2,100 each) that dominated every topic. After scrubbing that in `clean_text.py`, K=12 won — best coherence (−1.612), good diversity (0.87), balanced sizes (no catch-all, largest topic 16%), and it makes the one meaningful split that K=10 misses (separating *Machine Learning & Algorithms* from *Computing Systems & Security*). K=15+ started fragmenting (two materials topics, split health) with worse coherence and stability. Since coloring is off the table, K was chosen purely on interpretability.

The twelve topics, auto-named by anchor terms (rules in `topics.py`, so naming survives refits): Computing Systems & Security, Cell & Cancer Biology, STEM Education & Participation, Health & Behavioral Science, Energy/Power & Wireless, Machine Learning & Algorithms, Molecular & Structural Biology, Mathematics & Quantum Theory, Ocean/Climate & Environment, Mechanics/Structures & Control, Conferences & Workshops, Materials & Condensed Matter.

The interface. Color still means funding source. Below the year filter is a Topics list, sorted by prevalence with a size bar per row. Hovering a row highlights that topic on the map; because LDA gives every grant a topic distribution, the highlight is **weighted** — each dot's opacity scales with its relative p(topic | doc), so you see a topic's dense core and its faint penumbra rather than a hard in/out cut, and everything else grays out. Click pins a topic so you can move onto the map to inspect points; click it again, click another, or click empty space to release. The "keywords" checkbox reveals each topic's top terms on demand, and the hover tooltip now names a grant's dominant topic. It composes with the rest: a dot stays lit only if it's a topic member *and* inside the year window *and* in an active funding source.

Two notes. The 748 title-only grants mostly land in generic topics (Computing Systems, STEM Education) since a bare title carries little signal — expect some to re-sort once the real SPECTER2 run happens, though LDA itself is unaffected by the embedding choice. And the atlas is now its own validation: pin a topic and check whether its members pool into a contiguous region of the map. On the t-SNE preview several do (biology, ocean/climate), but the preview keys on lexical overlap; the SPECTER2+UMAP layout should make that pooling tighter. `pipeline.py` produces the identical `topics.json` + weights, so nothing in the UI changes when you swap in the real projection.

Grant atlas
Code · HTML
Pipeline
PY
Topics
PY
Scan k
PY
Build preview
PY
Clean text
PY

Remove the "Find" search box. Make the "Funding Source" menu/legend scrollable so that it can be made shorter. This way, we should have more space for the topics. Now most topics in the list are hidden.

Three changes: drop the Find card, cap the funding-source legend with its own scroll, and give that reclaimed space to the topics list (plus a safety scroll on the whole dock for short screens).

Edited 2 files, ran a command, viewed a file
Edited 2 files, ran a command, viewed a file

Now remove the search JS block:

Viewed 2 files, edited a file

Viewed 2 files, edited a file

Now the CSS: cap the legend with a scroll, enlarge the topics list, and let the whole dock scroll on short screens.

Edited 3 files, ran 2 commands, viewed a file
Edited 3 files, ran 2 commands, viewed a file

Done. The Find box is gone, the funding-source legend is now capped (about five rows) with its own scroll so it stays compact, and the topics list grew to fit all twelve rows. I also gave the whole dock a max height with a fallback scroll, so even on a short window every card — and every topic — stays reachable rather than getting pushed off the bottom.

Grant atlas
Code · HTML

Create a different visualization where the grants are organized in a circle packing fashion using the topics to group them. The grouping should be represented through a mix of placement (grants on the same topic should be close together) and line drawings that encapsulate the group of grants within the same topic.

A topic-packing view is a good complement to the atlas: same data, different question. I'll build it as a separate file — grants packed into tight per-topic clusters (circle packing), the clusters *placed* by their average position in the embedding so related topics sit near each other, and a smooth outline drawn around each cluster as the encapsulating "line drawing." I'll reuse the existing `grants_umap.json` and `topics.json`.

Created a file, ran a command
Created a file, ran a command

JS is valid. One fix: a pan-drag shouldn't clear a pinned topic (the trailing click after panning would). Let me guard it, matching the atlas behavior.

Edited 3 files, ran a command, read a file
Edited 3 files, ran a command, read a file

`topic_islands.html` — a separate view from the atlas, same underlying data. Each grant is a circle, packed tightly into its topic cluster (by dominant topic), so the grouping shows up two ways as you asked: through **placement** and through **outline**.

For placement, each topic is packed locally with d3's circle packing (`packSiblings`), and the twelve clusters are then positioned by their average location in the embedding — so topics that sit near each other in the atlas (e.g. the biology clusters, or systems/ML) land near each other here too, with a light force resolving overlaps so islands don't collide. For encapsulation, each cluster gets a smooth outline: I sample points around every circle's perimeter, take the convex

hull, and render it as a closed Catmull-Rom curve, so the boundary hugs the packed grants organically rather than as a rigid circle, with a faint fill and a topic-colored stroke plus a label.

Interactions mirror the atlas so the two feel like one tool: hover a topic in the list to highlight its island and fade the rest, click to pin, hover a circle for its details (including dominant topic). Two toggles — size by amount (re-packs with sqrt-scaled radii) and color by funding source (recolors the circles to the atlas palette while keeping the topic hulls and placement intact, so you can read topic and funder at once). Zoom/pan with reset-to-fit, white UI throughout.

Circles are colored by topic by default here, since the whole frame is topic-organized; flipping "Color by funding" gives you the cross-tab of topic-region against funder. It reads from the same `grants_umap.json` and `topics.json`, so when you run the real SPECTER2 + UMAP pipeline, the island *placement* updates automatically (the packing itself is projection-independent). One thing to expect: since islands are grouped by each grant's single dominant topic, a grant that straddles two topics lands wholly in one — the soft weights still live in the data if you ever want to express that here too.

Topic islands
Code · HTML

Can you create a hierarchical version of LDA? Let's discuss it before doing anything.

Happy to — and this is worth thinking through carefully, because "hierarchical LDA" names several quite different things, and the right one depends on what you want the hierarchy *for* in the atlas/islands context.

The first fork is what "hierarchical" refers to.

There's true nonparametric hierarchical topic modeling — hLDA with the nested Chinese restaurant process (Blei et al.), where documents pick a path down a topic tree and the tree structure and depth are learned, or the closely related nested HDP and Pachinko allocation. These give you a genuine generative tree where parent topics are coarse and children refine them. The problem for us is practical: there's no maintained hLDA in the sklearn/scipy world, gensim doesn't ship it, and the usable implementations (e.g. tomotopy for hLDA/PAM, or the original hlda samplers) are extra dependencies you'd install on your machine. They're also Gibbs-sampled and slower, and — the real catch — the learned tree is far less stable across runs than flat LDA, which for a labeled, colorable UI is a genuine cost.

The pragmatic alternative is a hierarchy *built on top of* flat LDA rather than a model that's natively hierarchical. Two sub-flavors:

Top-down (divisive): fit LDA at a small K for level 1, then within each level-1 topic's documents fit LDA again for level 2, recursively. This is easy with the code we already have, gives you clean parent→child document membership, and each subtree is independently interpretable. The downside is that sibling subtrees are modeled separately, so cross-branch relationships aren't

captured, and small level-1 topics may not have enough docs (or enough abstracts — remember 748 are title-only) to split well.

Bottom-up (agglomerative): fit one flat LDA at a fairly high K (say 15–20), then cluster those topics into a dendrogram using inter-topic distance — Jensen–Shannon divergence between their word distributions, or correlation of their document loadings. This is my instinct for your case, because it reuses the exact model we validated, it's fully deterministic, and it produces a tree whose leaves are the topics you already know. You'd get, e.g., a "Life Sciences" parent over Cell/Cancer, Molecular, and Health; a "Computing" parent over Systems, ML, and Wireless — emergent from the data rather than hand-grouped.

The second question is what the hierarchy should actually change in the product. Three options, increasingly ambitious: (a) it's purely a modeling artifact you inspect — I print/exported the tree and top terms, no UI change; (b) it becomes an interaction layer — the topic list in both tools becomes collapsible parent→child, and highlighting a parent lights up all its descendants' grants (this composes beautifully with the weighted-highlight and island designs we already have); (c) it drives layout — the islands view nests child clusters inside parent hulls (true circle-packing hierarchy, which is honestly the natural home for a topic tree). I'd lean toward (b) as the default with (c) as a strong follow-on, since nested hulls are exactly what a topic tree wants to look like.

Given all that, my recommendation: agglomerative on the existing K=12, clustered by Jensen–Shannon distance, cut to give ~4–5 parents over the 12 leaves, delivered first as an interaction layer (collapsible list + parent highlighting) and then, if you like it, as nested hulls in the islands. It's deterministic, needs no new dependencies, and keeps the labels and validation we've already done. I'd only reach for real nested-CRP hLDA if you specifically want the number of levels and the topic count itself to be learned rather than imposed — worth it as a research artifact, but a bigger, less stable lift.

A few things that would pin down the choice: how many levels do you actually want (just two — parents over the current topics — or deeper)? Should a document live at one leaf, or should the soft weights propagate up the tree so a grant can partially belong to several branches? And which K do you want the leaves to be — keep the validated 12, or refit higher (15–20) so there's more for the tree to organize? My default answers would be two levels, soft weights propagated upward (sum children to get parent membership, which keeps everything consistent with the weighted highlighting), and keep K=12 — but tell me where you'd differ.

Make a numbered short list of the choices I have to make before moving forward.

1. **Model type** — native nested-CRP hLDA (learns the tree + depth, new dependency, less stable) vs. a hierarchy built on flat LDA (deterministic, reuses our K=12).
2. **Build direction** (if flat-LDA-based) — bottom-up agglomerative (cluster the 12 topics into parents) vs. top-down divisive (split each topic into sub-topics).
3. **Number of levels** — two (parents over current topics) vs. deeper.
4. **Leaf K** — keep the validated 12 vs. refit higher (15–20) for the tree to organize.
5. **Number of parents** — e.g., ~4–5 over the 12 leaves (or where to cut the dendrogram).

6. **Inter-topic distance** (if agglomerative) — Jensen–Shannon between word distributions vs. correlation of document loadings.
7. **Document membership** — one leaf per grant vs. soft weights propagated up the tree.
8. **What it changes in the product** — inspect-only artifact / interaction layer (collapsible list + parent highlighting) / drives layout (nested hulls in the islands).

My suggested defaults: 1-flat-LDA, 2-agglomerative, 3-two levels, 4-keep 12, 5-~4–5 parents, 6-Jensen–Shannon, 7-soft weights, 8-interaction layer first, nested hulls as follow-on.

1. Hierarchy built (nested, not hLDA)
2. Top-down
3. Two levels
4. Leaf K = 25
5. Parents: around 6-8
6. Not agglomerative
7. One leaf per grant
8. Drives layout (nested hulls)

You've fully specified it, and the one place the choices need reconciling is worth stating up front: top-down + exactly 25 leaves + 6–8 parents means level 1 *is* the parent model (6–8 topics, one parent per grant by dominant level-1 topic), and each parent is then split by a second per-parent LDA into a number of children **proportional to the parent's size** (min 2 each), summing to 25. Not agglomerative, one hard path per grant, two levels. The flat atlas stays on the validated K=12; the hierarchy powers a new nested-hull islands view. Parents get distinct hues, children get lightness variants of the parent hue.

Let me first fit level 1 across $P \in \{6,7,8\}$ and read the parents before committing.

Created a file, ran a command
Created a file, ran a command

P=8 gives the cleanest, best-coherence taxonomy (−1.577) and sits in your 6–8 range, with a sensible split: Computing/Systems, Cell & Cancer Biology, Health & Society, Engineering Education, Energy/Devices, Ocean/Environment, Physics/Complex Systems, and Materials/Quantum — children allocated 4/4/3/3/3/3/3/2 = 25 by size. Locking that in and building the data.

Created a file, ran a command
Created a file, ran a command

The tree is clean and interpretable. Now the nested-hull view — grants packed into leaves, leaves packed within parents, both drawn as organic hulls (bold parent outline, thin child outlines), placed by embedding centroid at each level, with a collapsible parent→child list.

Ran 3 commands, created a file, viewed a file
Ran 3 commands, created a file, viewed a file

The existing file is a stale earlier draft with different markers. I'll remove it and write my vetted version.

Ran 2 commands, created a file
Ran 2 commands, created a file
Claude's response was interrupted.

1. Hierarchy built (nested, not hLDA)
2. Top-down
3. Two levels
4. Leaf K = 25
5. Parents: around 6-8
6. Not agglomerative
7. One leaf per grant
8. Drives layout - nested hulls

You've restated the same eight, and I've built exactly that — the nested-hull view is done. Let me finish wiring the hierarchy into the real pipeline and hand everything over. First, giving the parent-naming a proper home in the module so `pipeline.py` produces the same labels.

Edited 3 files, viewed a file
Edited 3 files, viewed a file

Now add `name_parent` + rules to the module top and import it:

Edited 2 files, viewed 2 files, ran a command
Edited 2 files, viewed 2 files, ran a command

Parent names resolve cleanly from the module now. Adding the hierarchy step to the real pipeline so the SPECTER2+UMAP run emits it too:

Edited 2 files, ran a command
Edited 2 files, ran a command

The outputs `pipeline.py` contains a parallel, mismatched hierarchy block from a stale write. Let me view it and reconcile to the actual `hier.py` API.

Edited 3 files, viewed a file, ran a command
Edited 3 files, viewed a file, ran a command

A couple of stale files from the parallel write are lingering. Cleaning up and doing a final data check on the hierarchical view:

Ran a command, read a file
Ran a command, read a file

Built to your eight choices. `topic_hierarchy.html` is the deliverable.

How each choice landed: it's a hierarchy **built on flat LDA** (no nested-CRP), **top-down** — level 1 is an 8-topic LDA giving the parents, then each parent's own documents get a second LDA. **Two levels**, with **exactly 25 leaves** allocated across the **8 parents proportional to size** (4/4/3/3/3/3/3/2), min 2 each. **Not agglomerative** — parents come first and are divided, never merged. **One hard leaf per grant** (dominant sub-topic within its dominant parent). And it **drives the layout as nested hulls**.

The tree that emerged: Computing & Systems (618) → Security / Networks / Software / Systems-Learning; Cell & Cancer Biology (489) → Cancer / Cells / DNA-Proteins / Drug-Compounds; Health & Society (424); Engineering Education & Convening (299); Energy, Devices & Wireless (292); Ocean, Climate & Environment (227); Physics & Complex Systems (188); Materials & Quantum Matter (139).

In the view, grants pack into their leaf, leaves pack within their parent, and parents are placed by their average embedding position — so related branches sit near each other. Each leaf gets a thin dashed hull, each parent a bold enclosing hull; parents carry a distinct hue and their children are lightness variants of it, so you read family by color and sub-topic by position. The tree panel on the left is collapsible (parent → children); hover highlights and fades the rest, click pins, and leaf labels appear when you focus a parent or zoom in. Size-by-amount and color-by-funding toggles work as in the islands view.

Scope note, so nothing surprises you: the flat atlas stays on the validated K=12 — this hierarchy powers the new nested-hull file only. The supporting code is `hier.py` (top-down fit + proportional allocation + naming) and `build_hier.py` (offline data build); `pipeline.py` now also emits `grants_hier.json` + `hier_topics.json` from the real SPECTER2+UMAP run, so the nested view updates automatically when you run it — the packing itself is projection-independent, but the parent/leaf *placement* will shift to the real embedding.

One heads-up from the build: a few stale files from an earlier parallel attempt were sitting in the workspace with a mismatched API; I reconciled `pipeline.py` to the real `hier.py` and removed the leftovers, so the outputs folder is now consistent.